全国高校AI Web开发挑战赛 · 技术文档

AI刁钻评委

面向大学生创新创业项目的 AI 智能评审与压力测试智能体

> 让你的项目先接受一次 AI 压力测试，再去面对真正的评委。

AI Native Web 应用 | Vue 3 + TypeScript | Agent 质询工作流 | 本地 RAG 检索增强 | Function Calling 工具

参赛团队 _____

学校 / 学院 _____

成员 / 指导教师 _____

日期 _____

ROADMAP

今天讲几件事

CONTENTS
2

01
是什么 & 为什么
项目定位 · 真实痛点 · 需求分析

02
怎么构建
系统架构 · AI 技术方案 · 核心功能实现

03
凭什么可信
测试验证 · 性能工程化 · 创新点

> 核心主张：**规则引擎管决策、LLM 管生成、RAG 管知识** —— 三层各司其职、失败自动降级， 让 AI 评审既"刁钻"又"不胡说"。

项目概述：一个"答辩前压力测试"工具

定位

AI Native Web 应用

从设计之初就以 AI 为核心：连续质询、回答分析、追问决策、锐评、风险累计、深度评审，全部由 AI 驱动。

场景

创新创业项目答辩准备

大学生把项目资料输入进来，AI 以资深评委身份压测：**回答完整 → 追问更深，有漏洞 → 针对缺口追问。**

形态

纯前端 Web 应用

Vue 3 + TypeScript + Vite + Pinia，路由懒加载；AI 能力层全部浏览器端调用，可部署于任意静态托管。

结果

可执行的改进清单

风险总结（按优先级） + 答辩材料清单 + 对标官方评分标准的改进路线，可直接导出 PDF。

22 轮	6 项	16 块	0 依赖
演示质询链（11 主线 + 11 追问）	AI 技术深度考察点全覆盖	赛题知识库检索单元	规则引擎本地运行，可离线演示

需求分析：答辩前的"没人帮挑毛病"

痛点	现状	方案映射
缺少预演	答辩前很少有机会被连续追问，临场才发现漏洞	AI 连续质询 + 追问链，模拟真实评审节奏
反馈笼统	"再想想"式意见多，缺逐条可执行的依据	每条问题给出 为什么是问题 / 评委怎么追问 / 建议补什么
数据风险	为凑指标虚构数据，被评委一句话问穿	数据红线内置：缺失即标注"待验证"，不替你编
标准不对齐	凭感觉准备，不清楚官方评分维度	RAG 注入官方赛题评分标准，锐评对标考察点

> 目标用户：参加创新创业类竞赛的高校学生（1–3 人团队），核心诉求是**答辩前低成本发现漏洞、拿到可执行的改进清单。**

功能全景：一条完整的"输入 → 压测 → 报告"链路

F1
项目库
示例项目 + 自定义项目，localStorage 持久化，不上传

F2
AI 连续质询
11 主线题 + 追问链，视觉类项目自动追加专项题

F3
AI 锐评
规则判定骨架 + LLM 深度锐评，流式打字机呈现

F4
实时风险面板
风险按优先级累计，评审维度得分条实时更新

F5
模拟评审报告
结论 → 维度分 → 风险 → 建议 → 材料清单，可导出 PDF

F6
AI 深度评审报告
Agent 自主调用工具，对标官方评分标准的深度报告

F7
语音输入
Web Speech API，普通话语音转文字填入回答

F8
会话恢复
刷新不丢：历史输入持久化，重放恢复质询现场

系统架构：前端为主体，AI 能力层浏览器端运行

VIEWS · 路由懒加载（代码分割）

Home 首页 | Library 项目库 | Input 输入项目 | Review 质询三栏 | Report 报告

↑ 组件调用 ↓

STORES · Pinia 状态管理

projects（项目库持久化） | session（会话 / 草稿 / 重放恢复）

↑ 核心逻辑 ↓

ENGINE + DATA · 质询引擎（纯逻辑，Node 可测）

追问链组装 | 回答分析 | 评分 | 风险累计 | 报告 | 题库 / 词典 / 演示项目 | 赛题知识库（16 块）

↑ AI 能力层 ↓

SERVICES · AI 能力（全部浏览器端调用，失败自动降级）

llm 流式客户端 | aiReview 锐评增强 | rag 本地检索 | tools Function Calling | deepReport 深度评审 | voice 语音 | safety 安全

AI 技术方案：三层各司其职，降级不降智

层 1 · 规则引擎

决策骨架

追问链选择（回答完整走 deepQ 再压一层 / 有漏洞走 gapQ 针对缺口）、证据识别、维度评分、风险累计。**确定性、可解释、本地零依赖。**

层 2 · LLM

内容增强

锐评总结、评委追问建议、深度评审报告。流式输出 + JSON 稳定化； **未配置 Key 或调用失败 → 自动降级为纯规则引擎。**

层 3 · RAG

知识注入

官方赛题与评分标准知识库，本地词频检索；锐评前自动注入相关考察点， 让评审意见**对齐真实评分维度**而非泛泛而谈。

提交回答
输入过滤 → 长度校验

→

规则判定
证据识别 · 问题清单 · 追问决策

→

RAG 检索
问题+回答 → 知识库 Top-3

→

LLM 增强
流式锐评 · 追问建议

大模型集成：选型、流式、Prompt 与稳定化

选型理由

OpenAI 兼容抽象层

统一 chat/completions 接口，baseURL 可配，**DeepSeek / 智谱 / 通义 / Moonshot 一键切换**； 评审环境可选用任意国内可用模型，规避网络依赖。

集成方式

SSE 流式输出

深度锐评逐 token 渲染（打字机效果），思考过程可视化； 非流式 JSON 模式用于需要结构化解析的场景（response_format）。

Prompt 设计

角色 + 铁律 + 结构

资深评审官角色设定；铁律（不虚构数据 / 只评当前轮 / 输出合法 JSON）； 输出结构固定为 summary + asks，解析失败自动降级。

稳定可控

超时 + 重试 + 降级

45s 超时、网络错误自动重试一次、失败返回 null → 界面回退规则引擎版本， 演示现场不会因 API 抖动而中断。

> 输出稳定性策略：**温度 0.35 + JSON 模式 + 结构校验 + 降级兜底**，同一回答多次生成结构一致。

RAG 检索增强：让锐评对齐官方评分标准

知识库

16 块官方赛题知识

命题背景 · 10 个推荐方向 · 评分权重 · AI 深度六项考察点 · 创新性细则 · 前端工程细则 · 提交要求 · 技术文档章节 · 数据红线，全部提炼自官方赛题文档。

检索策略

本地词频检索（零依赖）

中文 2-gram + 英文单词切词、词频打分、关键词预定义加权； **完全浏览器端运行**：项目内容与知识库不外传，隐私安全。

注入时机

每轮锐评自动注入

以"问题 + 回答 + 判定"为 query 检索 Top-3 片段，注入 Prompt 的【赛题与评分标准参考】节，让 LLM 在真实考察点语境下给意见。

验证

6 组查询全中

评分标准 / 提交要求 / RAG 方向 / 技术栈 / 数据红线 / 文档要求 6 类查询全部命中正确知识块（检索单测通过）。

Agent 工作流：AI 深度评审报告

收集材料
项目资料 + 全部质询轮次 + 规则引擎汇总
→
Agent 决策
LLM 自主决定调用哪些工具
→
执行工具
查知识库 / 按评分表评估
→
流式成稿
深度评审报告

工具	说明	对应评分考察点
search_contest_kb	检索官方赛题知识库（评分权重 / 考察点）	RAG 技术应用 · 工具集成
score_against_standards	按官方评分维度返回权重与评估依据	Agent 自主决策 · 输出可控

> 两轮对话模式：第一轮带工具列表让模型**自主规划**，执行工具后把结果送回， 第二轮流式生成对标官方评分标准的报告（总体判断 → 逐维分析 → 三大风险 → 答辩清单）。

多模态：语音输入（Web Speech API）

实现

浏览器原生语音识别

SpeechRecognition / webkitSpeechRecognition，zh-CN 普通话， 连续识别 + 中间结果；识别文本自动追加到回答输入框。

降级

能力检测

不支持语音识别的浏览器自动隐藏麦克风按钮，不影响核心质询流程； 识别失败提示错误并停止，可重试。

体验

贴合答辩场景

选手可以**用嘴演练答辩**，语音转文字直接提交， 比打字更接近真实答辩节奏，AI 锐评照常给出追问建议。

成本

零第三方依赖

不引入语音服务 SDK，无额外费用、无数据外传； 与"本地优先"的产品原则一致。

多模态设计取舍：现阶段以**语音输入**作为多模态入口（浏览器原生、零成本）， 为后续接入答辩录音分析、上传材料 OCR 等模态预留扩展位。

SAFETY

AI 安全与可控性：能降级、能解释、不越界

1
输入过滤
暴力 / 违法 / 攻击性内容本地拦截；超长输入（>6000 字）限制

2
降级兜底
LLM 未配置 / 超时 / 失败 → 自动回退规则引擎，功能不中断

3
可解释性
锐评逐条给出 为什么是问题 / 评委怎么追问 / 建议补什么

4
数据红线
不虚构数据；缺失标注"待验证"；报告声明非官方评审

- ✓ **密钥不入库**：API Key 走环境变量（.env 已 gitignore），提交仓库零泄露。
- ✓ **本地优先**：项目资料、知识库、检索全部浏览器端完成，无服务器中间层。
- ✓ **模拟评审声明**：界面、报告、PDF 均标注"不代表官方评审 / 官方评分"。

质询引擎：不是随机提问，是追问链

Q **资料驱动出题**：示例项目 11 道主线题按 *创新性 → 技术实现 → AI 能力 → 视觉专项 → 数据验证 → 异常鲁棒 → 通信 → 实用价值* 推进； 自定义项目用通用链，检测到视觉关键词自动插入 4 道视觉专项题。

A **回答分析**：8 条规则 + 题目专属规则 —— 信息量不足 / 关键证据缺失 / 隐含问题被绕开 / 参数缺失 / 有结论无量化 / 含糊表述 / 绝对化表述 / 缺失效处理。

↳ **追问决策**：回答完整（证据覆盖 ≥ 2/3 且无高优问题）→ **deepQ 再压一层**； 有漏洞 → **gapQ 针对缺口追问**；每题最多 1 轮追问后自动推进主题。

R **风险累计与评分**：问题按 6 类风险聚合（证据不足 / 逻辑漏洞 / 参数缺失 / 技术风险 / 数据风险 / 发现的问题）， 维度分按题目映射实时增减。

质询现场：三栏布局，信息实时联动

左栏

项目信息 + 进度

项目资料逐项展示；质询进度（主线 / 轮次 / 发现问题）；追问链可视化（✓ 已完成 / ▶ 当前）。

中栏

当前问题 + 回答 + 锐评

CURRENT QUESTION 卡片（评委关注点 + 完整回答应包含）； 手动 / 模拟 / 语音三种回答方式；AI 锐评流式渲染；往轮记录可折叠回看。

右栏

实时风险面板

风险按优先级实时累计（高 / 中 / 低 + 出现次数）；模拟评审维度得分条随回答即时变化。

交互细节：**模拟回答连续点击**即可展示追问链全流程（演示友好）； 任意时刻"结束并出报告"；**会话持久化**刷新不丢（历史输入重放恢复）。

报告体系：规则报告 + AI 深度报告，均可导出 PDF

规则引擎报告

确定性输出

结论 → 模拟评审维度（非官方评分）→ 风险总结（按优先级）→ 改进建议（必补 / 建议 / 加分）→ 建议补充材料 → 质询回合一览 → 视觉专项清单。

AI 深度报告

对标评分标准

Agent 调用工具核对官方考察点后生成：总体判断 → 逐维分析（当前状态 / 差距 / 怎么补）→ 三大最紧迫风险 → 答辩准备清单。

打印 / 导出 PDF
浏览器打印对话框另存为 PDF

→

A4 浅色版式
打印样式独立重写：白底深字、表格卡片不跨页

→

页眉声明
项目名 + 场景 + 免责声明自动带入

演示场景：模拟回答 ×3 → 结束出报告 → 导出 PDF，**3 分钟内跑完一次完整闭环**。

测试与验证：不是"应该能跑"，是跑过并量化

测试	命令	结果
质询引擎单测	npm run test:engine	11 主线 + 11 追问共 22 轮完整跑通，风险 / 维度分 / 报告 / 校验全过
RAG 检索单测	npm run test:rag	6 组查询全部命中正确知识块
端到端点击流	npm run test:e2e	无头 Edge 真实点击：首页 → 快速体验 → 模拟回答×3 → 报告 → 返回追加，6 项断言全过
类型与规范	npm run build / lint	vue-tsc 严格模式 0 错误；ESLint 0 错误；Prettier 统一

22 轮
引擎完整链

6 / 6
E2E 断言

0
lint / 类型错误

前端工程化：严格类型 + 路由懒加载 + 代码分割

TypeScript 严格

类型即文档

noUnusedLocals / noUnusedParameters / erasableSyntaxOnly 全开， 全量类型定义（Question / Session / Critique / Report…），引擎层 Node 可直接运行测试。

性能优化

懒加载 + 分包

路由级动态 import（5 视图独立 chunk）；vue 全家桶 vendor 分包； 构建产物 gzip：vendor 34 KB + 主包 42 KB + 视图 1.5–6 KB。

规范

ESLint + Prettier

vue-ts 官方 ESLint 配置 + Prettier 统一格式，npm run lint / format 一键执行。

工程结构

分层清晰

engine（纯逻辑）/ data / services / stores / views / components 六层分离， 引擎与 UI 完全解耦，可独立测试、可复用。

架构收益：**质询引擎不依赖 DOM、可在 Node 直接跑**——单测、文档、未来迁移都不受界面层约束。

创新点：技术 × 应用 × 交互

技术 01

规则引擎 × LLM 混合评审

决策与生成分离：追问链、评分、风险由确定性规则引擎负责， LLM 只做内容增强——**可解释、防幻觉、可降级**。

技术 02

本地 RAG 对齐官方标准

赛题评分标准做知识库、浏览器端词频检索、每轮锐评自动注入， **零依赖实现检索增强生成**，评审意见落到官方考察点。

技术 03

Agent 自主工具调用

深度评审报告由 LLM 自主决定调用"查知识库 / 评分表评估"， Function Calling 两轮对话，真实 Agent 工作流。

应用 01

答辩前压力测试

把"没人帮挑毛病"变成"AI 连续压测"， 输出**逐条可执行的改进清单**，直接进答辩准备。

应用 02

数据红线产品化

不虚构数据写入产品铁律：缺失即标注"待验证"， 引导选手用"目标 / 计划测试"的正确答法。

应用 03

追问链交互

回答完整就深入一层、有漏洞就针对缺口—— **不是随机提问**，模拟真实评委的追打节奏。

总结与展望

总结

一个可验证的 AI Native 应用

从真实痛点出发，用 Vue 3 + TS 构建完整前端工程， 以"规则引擎 + LLM + RAG"三层架构实现 AI 深度集成， 配齐测试、性能与安全工程化，所有结论可复现。

展望 1

多模态扩展

答辩录音 / 录屏分析（转写 + 追问）、上传材料 OCR 提取证据， 从语音输入扩展为"答辩全流程 AI 教练"。

展望 2

知识库可插拔

赛题知识库按 JSON 配置化，可切换不同比赛 / 评审标准， 从一个比赛工具变成通用"答辩教练"平台。

展望 3

端侧推理

接入 Transformers.js / WebGPU 做本地小模型兜底， 进一步兑现"端云协同"的技术路线。

> 社会价值：降低大学生创新创业项目的"答辩信息差"—— 让每一支队伍在上场前，都有一位**不辞辛劳、不讲情面、不虚构数据**的 AI 刁钻评委。